

# Programmation C

## 8 – Compléments sur les déclarations

Alain CROUZIL, Emmanuel LAVINAL

`alain.crouzil@irit.fr, emmanuel.lavinal@irit.fr`

Département d'Informatique  
Faculté Sciences et Ingénierie  
Université de Toulouse

Licence  
Semestre pair 2025-2026

 | Département  
Informatique

 Faculté  
sciences et  
ingénierie  
Université  
de Toulouse

# Sommaire

1 Qualifieur `const`

2 Classe `static`

3 Classe `extern`

# Sommaire

1 Qualifieur `const`

2 Classe `static`

3 Classe `extern`

# Qualifieur `const` (1/3)

## Rôle

- **`const`** permet de préciser que la valeur d'une variable ne doit pas changer lors de l'exécution du programme.
- Les éventuelles instructions permettant la modification de la valeur de cette variable sont signalées par le compilateur sous la forme d'erreurs ou d'avertissements (*warnings*).

## Exemples

- **`const int`** `a=12`; déclare un entier constant de valeur 12. La valeur de `a` ne peut pas être changée par la suite :

```
a=20; /* produit une erreur */
```

```
a++; /* produit une erreur */
```

```
int *p;
```

```
p=&a; /* produit un avertissement */
```

```
*p=20;
```

## Qualifieur `const` (2/3)

### Exemples (suite)

- Avec les pointeurs :

```
const char * pc1; /* la zone pointée est constante, le pointeur est modifiable */
```

```
char c;
```

```
char * const pc2=&c; /* la zone pointée est modifiable, le pointeur est constant */
```

```
const char * const pc3=&c; /* la zone pointée et le pointeur sont constants */
```

# Qualifieur `const` (3/3)

## Exemples (fin)

- Avec les variables structurées :

*Exemple 1 :*

```
struct sDate
{
    unsigned int jour;
    unsigned int mois;
    unsigned int annee;
};
const struct sDate revolution={14,7,1789}; /* tous les champs sont constants */
```

*Exemple 2 :*

```
struct sDate
{
    unsigned int jour;
    const unsigned int mois;
    const unsigned int annee;
};
struct sDate revolution={14,7,1789}; /* le champ jour est modifiable */
```

# Sommaire

1 Qualifieur `const`

2 Classe `static`

3 Classe `extern`

# Classe static

## Rôles

- Pour une variable globale : elle n'est pas accessible depuis d'autres fichiers (cachée).
- Pour une fonction : elle n'est pas accessible depuis d'autres fichiers (cachée).
- Pour une variable locale : la variable est rémanente, sa valeur est conservée d'un appel à l'autre.

## Exemple de variable rémanente

Si l'on définit la fonction suivante :

```
void f(void)  
{  
    static int i=0;  
    i++;  
    printf("appel_numéro : %d\n",i);  
}
```

les deux appels : f(); f();

produisent l'affichage de :

appel numéro : 1

appel numéro : 2

# Sommaire

1 Qualifieur `const`

2 Classe `static`

3 Classe `extern`

# Classe extern

## Rôles

- Pour une variable : elle est définie (déclarée avec réservation d'un emplacement en mémoire) ailleurs, en général dans un autre fichier.
- Pour une déclaration de fonction : elle est définie ailleurs, généralement dans un autre fichier.

## Exemples

```
extern int a;  
extern int f(int n);
```

La variable `a` et la fonction `f` sont utilisables, mais, pour l'édition de liens, il faut qu'elles soient définies ailleurs et visibles.